

Progressive Retrieval Attention on NVIDIA TensorRT-LLM: Semantic Memory Contracts, E0 Serving Evidence, and the Native Attachment Boundary

Jorge Simão

September 2026

Abstract

We ask what Progressive Retrieval Attention (PRA) improves in front of TensorRT-LLM and which gains come from session-aware Selected Context rather than native engine integration. TensorRT-LLM already owns paged K/V, prefix reuse, in-flight batching, and connector lifecycles. We implement an OpenAI-compatible Selected Context path (E0), semantic resource identities, authorization and cleanup contracts, and a capability gate for Native Memory (E2). A pinned source audit finds no public arbitrary non-prefix attachment operation; a correct E2 implementation would cross request conversion, scheduler metadata, cache management, connector state, and the fused attention ABI.

On an RTX 5060 Laptop GPU, a 1,240-request load sweep through concurrency sixteen reaches 81.9 requests/s for independent selected inputs versus 53.6 for full context. Matched TTFT p99 is 50.5 versus 157.8 ms. Across a three-point pressure curve, selected throughput remains 78.0–81.9 requests/s as full visible context grows to 1,276 tokens. A separate 360-request natural cohort covers QASPER, HotpotQA, and 2Wiki. A 120-request two-turn session cohort reduces turn-two prompt tokens from 797 to 658 and newly visible resource tokens from 52 to 26, while all 20 paired outputs match.

These are transport and serving results, not task-quality or native-memory claims. TinyLlama obtains 0/20 exact-code success in both session conditions, and the installed public API cannot execute E2. Page-table splicing alone would also corrupt sequential geometry and cache-write offsets. The recommended path for TensorRT-LLM 1.2.1 is therefore session-aware E0 with ordinary prefix reuse. Native PRA remains an upstream integration contract whose quality, memory, and economic effects are NOT MEASURED.

1 Qualification Snapshot

Engine question. Once TensorRT-LLM already supplies paged K/V, prefix caching, and in-flight batching, does PRA provide value beyond a strong selected-text baseline? Table 1 separates the answer supported by this paper from the native experiment that remains blocked.

Recommendation. Use E0 selected text on the tested stack when frozen retrieval can remove substantial distractor context. Do not configure or describe this adapter as native PRA. Promotion to E2 requires an official non-prefix attention seam and a selector-frozen E0/E2 parity experiment; E3 additionally requires scheduler-owned placement, sharing, eviction, and accounting.

Primer. E0 changes which source tokens enter an otherwise ordinary prompt. E1 carries PRA identities, records, and fallback policy through the runtime but does not attach selected K/V. E2 removes selected source text from the visible prompt and exposes its detached native K/V directly to attention. E3 gives the engine scheduler physical ownership of those selected K/V pages. This paper measures the first step and specifies the boundary to the latter two.

Table 1: Standard PRA engine result block. NOT MEASURED is an explicit evidence status, not an assumed zero.

Field	Result
Validated / observed level	E0 validated; E2 and E3 not observed
Model / hardware	TinyLlama-1.1B; RTX 5060 Laptop GPU, 8,151 MiB
Quality	Controlled success 1.00; natural token F1 0.069–0.105 for E0 selected evidence
Visible context	67 selected tokens versus 1,040–1,276 full-context tokens
Loaded serving	78.0–81.9 selected requests/s at concurrency 16; 50.5–86.0 ms TTFT p99
Native K/V tokens and bytes	NOT MEASURED: no public non-prefix attachment operation
Transfer / E2 reuse / E3 scheduling	NOT MEASURED
Evidence tier	Controlled transport, 360-request natural QA, and live installed-source audit

2 Introduction

This paper asks what PRA improves in front of TensorRT-LLM before arbitrary non-prefix K/V attachment exists. The companion runtime paper owns the common deployment and evidence contract [1]; this paper separates session-aware selected-text gains from unimplemented native claims.

Prefix caching asks whether a request begins with tokens that an engine has already processed. PRA asks which independently retained resource is relevant to the current query. The two mechanisms can cooperate, but only if three identities remain distinct:

$$I_{\text{prefix}} \neq I_{\text{PRA}} \neq I_{\text{physical block}}.$$

Conflating them creates two failure modes. A selected resource can leak into a later request through ordinary prefix reuse, or the same K/V can be included twice through both sequential and PRA paths. TensorRT-LLM is a demanding target because it already owns an optimized physical K/V system. A useful integration must add semantic identity without replacing that system.

Selected Context session deduplication is owned by the shared PRA runtime; engine-native prefix caching is measured independently. TensorRT-LLM block reuse therefore reports physical sequential-prefix reuse only. It neither establishes that a semantic resource is still model-visible nor substitutes for the blocked non-prefix native attachment operation.

This paper contributes three bounded results. First, it implements the shared runtime and safety contracts needed by a TensorRT-LLM adapter. Second, it audits the installed 1.2.1 runtime down to request, paged-cache-manager, and connector interfaces. Third, it measures a reproducible E0 selected-text baseline under prefix reuse and in-flight batching. The native result is a precise negative API finding: the installed public surface can load sequential request pages, but cannot expose an arbitrary set of selected non-prefix pages to attention.

3 Execution Levels and Claim Boundary

We use the shared PRA execution-level vocabulary.

E0: Selected source text is rendered into the visible prompt. Retrieval and semantic resource identity exist, but the engine performs ordinary sequential prefill.

E1: The runtime carries PRA logical identities, typed records, routing decisions, and explicit fallback policy, but attention remains ordinary.

E2: Attention consumes selected native K/V without rendering the source text into the prompt, with matched geometry and one joint normalization.

E3: The engine scheduler owns physical placement, sharing, promotion, eviction, and accounting for the selected native K/V.

The live measurements in this paper are E0. The native classes are executable contracts and tested lifecycle machinery, not evidence that TensorRT-LLM 1.2.1 performs E2 attachment.

4 TensorRT-LLM Mapping

4.1 What the engine already owns

TensorRT-LLM owns model execution, paged K/V allocation, prefix lookup, retention, in-flight batching, and connector transfer. PRA owns logical records, routing, authorization, selected intervals, session/task policy, and HOT/WARM/COLD/SOURCE retention decisions. Physical block identifiers are derived state and are never stable resource identifiers.

4.2 Implemented adapter surface

`src/prt_tensorrt_llm/adapter.py` provides `TensorRTLLMEngineAdapter`. Without a native executor, it reports E0 and uses the supported `trtllm-serve` HTTP facade. With an explicitly installed executor it can report E2, making capability elevation a dependency injection decision rather than an inference from package availability.

`src/prt_tensorrt_llm/native.py` defines four pieces needed by a future engine seam:

1. a topology fingerprint over model revision, tensor/pipeline parallelism, K/V precision, and block size;
2. immutable logical-to-physical block handles;
3. a connector-facing store protocol with pin and unpin operations; and
4. a request attachment manager enforcing authorization, tenant isolation, exactly-once exposure, cleanup, and exclusion from ordinary prefix pools.

Tests exercise cross-tenant rejection, topology mismatch, duplicate attachment, pinned-resource removal, and cleanup after request completion.

4.3 Public API audit

The live package audit records `Request.cache_salt_id` and `kv_cache_retention_config`; paged manager operations including `get_cache_block_ids`, `pin_blocks`, and `unpin_blocks_by_id`; and the official worker/scheduler connector interfaces. These are sufficient for sequential cache reuse, retention, and external page movement. They are not sufficient for E2. Neither the request nor cache manager exposes a field or operation such as selected non-prefix block IDs or `attach_nonprefix_blocks`.

The official connector can load pages associated with request-token ranges. Treating those pages as arbitrary semantic memory would silently turn PRA into prefix restoration. A minimal E2 extension must instead carry an authorized selected-block set into request attention metadata and combine local and memory scores in one softmax. That source-level seam is future work.

4.4 Source-level seam feasibility

We next audited the installed Python sources rather than inferring feasibility from API names. The artifact records SHA-256 hashes so that the line references below identify one exact 1.2.1 installation. Request conversion begins at `llm_request.py:731` and carries `cache_salt_id` at line 824, but carries no selected non-prefix block identity. The model engine calls `attn_metadata.prepare()` at `model_engine.py:1512`. During that preparation, `trtllm.py:990` copies the block offsets owned by each request ID. The same backend passes one `kv_cache.block_offsets` tensor into `thop.attention` at line 500. The metadata contract has neither a second memory table nor independent local and memory lengths.

The cache manager does provide pin and unpin operations (`resource_manager.py:1340` and `882`), and the connector exposes layer-load synchronization (`kv_cache_connector.py:123--130`). Those are useful physical primitives. They do not answer how independently retained pages enter attention without changing the request’s sequential cache layout. Table 2 summarizes the resulting patch surface.

Table 2: Installed-source feasibility audit. All five surfaces must agree for native E2; connector availability alone is insufficient.

Surface	Installed 1.2.1	Native PRA requirement
Request ABI	No selected non-prefix field	Authorized external block identities
Cache lifecycle	Request-owned table; pin/unpin	Persistent independent pages
Attention metadata	One local table and K/V length	Separate local/memory geometry
Fused operator	No native-memory input	One joint local+memory softmax
Connector	Loads request-token ranges	Topology-bound semantic pages

Three apparent shortcuts fail the semantic contract. Prepending selected pages to the ordinary block table shifts K/V length, causal geometry, and write locations. Reporting connector restoration as E2 confuses prefix-range movement with query-addressed attention. Reusing encoder-decoder cross attention would require modules and projection geometry absent from the tested decoder-only checkpoint. We therefore stop at E0 for TensorRT-LLM 1.2.1 rather than maintain a version-fragile fork across request bindings, scheduling, metadata, CUDA-graph buffers, and the fused operator. This is a version-scoped maintainability decision, not evidence that native PRA is impossible in a future supported API.

5 Experimental Method

5.1 Environment

The live system uses TensorRT-LLM 1.2.1, TensorRT 10.14.1.48, PyTorch 2.9.1, and the PyTorch backend on an NVIDIA GeForce RTX 5060 Laptop GPU (compute capability 12.0, 8,151 MiB). The model is TinyLlama/TinyLlama-1.1B-Chat-v1.0. The server uses a 2,048-token maximum sequence, 2,048 scheduled tokens, batch size eight, 32-token K/V blocks, and a 0.30 free-memory fraction. Reported device use is 4,914 MiB during the warm E0 sweep.

5.2 Frozen conditions

The controlled resource states that the answer is `PRA_EVIDENCE_4821`. Conditions are no evidence, prefix only, selected text, prefix plus selected text, and bounded full context. The full context contains the same evidence plus six distractor resources and remains within TinyLlama’s context limit. Greedy decoding emits at most 24 tokens. Each one-shot condition has five repetitions. The concurrency experiment uses five waves at concurrency 1, 2, 4, and 8 for shared and independent resources. Every logical resource is explicitly primed outside the timed window, so the concurrency table is warm steady-state. E0 and full-context conditions use the same evidence; no independent retrieval decision is made. The expanded sweep raises batch size to 16, scheduled tokens to 4,096, and the free-memory fraction to 0.50; it uses ten waves at concurrency 1, 2, 4, 8, and 16 for each context and resource-sharing condition.

5.3 Metrics and scope

We report visible and cached tokens, controlled answer success, TTFT, completion latency, requests/s, output tokens/s, and HBM snapshots. Five waves support smoke-level tails, not production percentile claims. Because E2 is blocked, native K/V bytes, connector transfer, E0/E2 parity, and disaggregated PRA transfer are intentionally absent.

The repeated-session extension freezes the logical selections to A/B/C on turn one and B/D on turn two. The no-reuse condition serializes B again; the session-aware condition retains B at its original message location and serializes only D. Five seeds and four repetitions produce 20 paired second turns per condition. We report logical resource-token accounting, actual prompt usage, engine-reported cached tokens, TTFT,

ITL, completion latency, exact task success, and paired output equivalence. These measurements keep logical reuse, prefix-cache compute reuse, and native-state reuse in separate fields.

6 Results

6.1 Capability gates

Table 3: Implemented capability and evidence status. Contract means the PRA-side lifecycle is executable and tested but not attached by the live TensorRT-LLM scheduler.

Capability	Status	Evidence
OpenAI selected-text E0	Ready	25 live generations
Prefix reuse and cache salt	Ready	request fields and token counters
Paged manager and connector	Available	live package introspection
PRA lifecycle/isolation	Contract ready	focused unit tests
Native selected non-prefix E2	Blocked	no public attachment hook
Disaggregated PRA E2	Not run	depends on native attachment

6.2 One-shot E0 behavior

Table 4: Controlled E0 serving smoke. Visible is mean prompt tokens; times are milliseconds. Cold is the first request and warm is the mean of repetitions 2–5.

Condition	Visible	Success	Cold TTFT	Warm TTFT	Latency p50
no_prefix_no_pra	29	0.00	183.42	26.1	186.0
prefix_only	536	0.00	25.85	35.1	193.1
pra_only	67	1.00	33.98	26.8	157.8
prefix_plus_pra	574	1.00	32.33	27.1	189.2
full_context	1040	1.00	26.18	21.8	155.9

The controls never emit the target code, while all evidence-bearing conditions do. Selected text reduces visible input by 93.6% relative to bounded full context (67 versus 1,040 tokens). Once both prompts are cached, the small smoke does not establish a TTFT advantage: warm means are 26.8 and 21.8 ms. The main benefit appears when multiple requests compete for prefill and decode capacity.

6.3 Session-aware Selected Context

Table 5: Live two-turn TensorRT-LLM session cohort, 20 measured second turns per condition. Prompt and cached values are engine-reported token means. Equivalence compares each output with the matched Selected Context run that re-serializes B. Native Memory is NOT MEASURED.

Condition	Prompt	Cached	Turn-2 new	Logical reuse	Equiv.	Exact
Full context	1,075	1,074	102	0	0.00	0.00
Selected Context, no reuse	797	796	52	0	1.00	0.00
Session-aware Selected Context	658	657	26	26	1.00	0.00

The shared realization ledger removes the second copy of B without moving or rewriting its first-turn occurrence. This reduces the actual second-turn prompt by 139 tokens (17.4%) relative to Selected Context without logical reuse and by 417 tokens (38.8%) relative to this repeated full-context control. All 20 session-aware outputs exactly match their no-reuse counterparts. That is a behavioral-preservation result, not answer

correctness: TinyLlama begins an explanatory response and never emits both requested codes within the bounded decode in any condition. The cohort therefore supports transport equivalence and token reduction only.

With the prompts already warm from an immediately preceding audit, the engine reports nearly every prompt token cached. Median TTFT is 20.3 ms without logical reuse and 21.3 ms with it; p95 is 24.3 and 29.3 ms. These small mixed differences do not support a latency advantage. They do show why prefix telemetry cannot be used as the logical correctness criterion: both prompts are physically warm, but only the shared runtime knows that B need not be serialized again. The raw rows are in `session_aware_selected_context_live.json`.

6.4 Warm in-flight batching

Table 6: Warm E0 concurrency endpoints. Every row has controlled answer success 1.00. Cached is the engine-reported mean cached-token count.

Transport / resources	C	req/s	TTFT p99	completion p99	cached
selected / shared	1	6.5	24.3	158.5	66
selected / shared	8	41.7	57.9	194.9	64
selected / independent	1	6.3	35.4	166.6	66
selected / independent	8	43.9	42.9	184.2	66
full / shared	1	6.4	26.7	159.0	1039
full / shared	8	27.2	155.5	302.8	1039
full / independent	1	6.5	22.5	156.4	1039
full / independent	8	29.0	139.7	282.6	1039

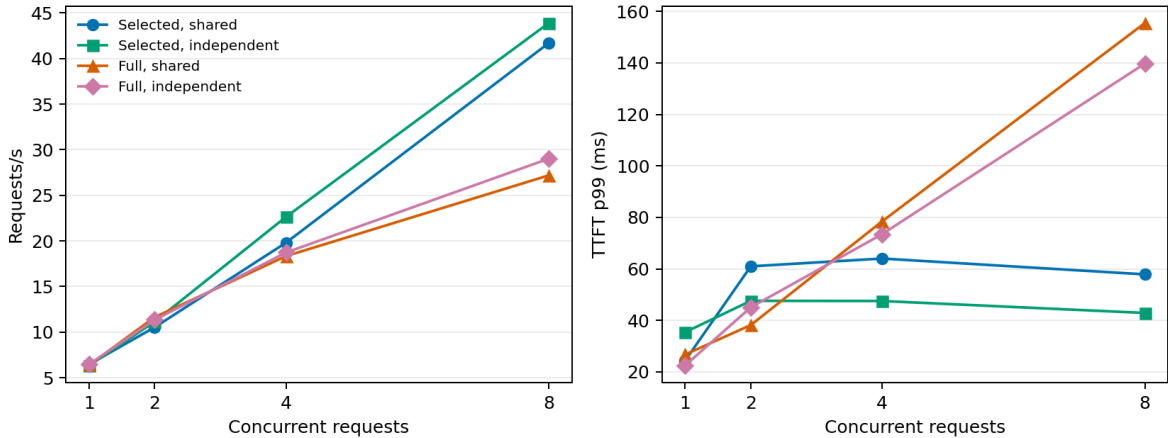


Figure 1: Warm E0 throughput and TTFT tail across in-flight load. These curves compare selected text with bounded full context; they are not native E2 results.

At concurrency eight with a shared resource, selected text reaches 41.7 requests/s versus 27.2 for full context, a $1.53\times$ ratio. TTFT p99 falls from 155.5 to 57.9 ms, and completion p99 from 302.8 to 194.9 ms. The independent-resource comparison is similar: 43.9 versus 29.0 requests/s and 42.9 versus 139.7 ms TTFT p99. Engine-reported cached input is about 64–66 tokens for selected text and 1,039 for full context. Device memory remains 4,914 MiB before and after each group, so this bounded E0 test shows scheduling and token-work differences, not a measurable HBM residency reduction.

6.5 Expanded load and natural-input validation

Table 7: Expanded warm E0 endpoints. Each row contains ten measured waves; the same target record appears in selected and full context.

Workload	Context	C	Succ.	Req/s	Out tok/s	TTFT p50	TTFT p99	ITL p99
Shared	Selected	1	1.00	6.41	128.1	23.2	32.0	7.0
Shared	Selected	8	1.00	40.18	803.5	41.5	132.8	7.8
Shared	Selected	16	1.00	79.61	1592.2	46.6	58.4	7.8
Independent	Selected	1	1.00	6.53	130.5	20.4	24.8	7.0
Independent	Selected	8	1.00	40.72	814.4	50.4	60.9	8.0
Independent	Selected	16	1.00	81.92	1638.5	43.4	50.5	7.8
Shared	Full	1	1.00	6.10	121.9	27.6	35.0	7.4
Shared	Full	8	1.00	34.65	693.0	67.0	80.3	8.8
Shared	Full	16	1.00	54.77	1095.5	100.7	181.8	11.4
Independent	Full	1	1.00	6.33	126.6	22.9	25.2	7.4
Independent	Full	8	1.00	30.21	604.3	73.5	511.1	9.2
Independent	Full	16	1.00	53.60	1072.0	114.7	157.8	10.9

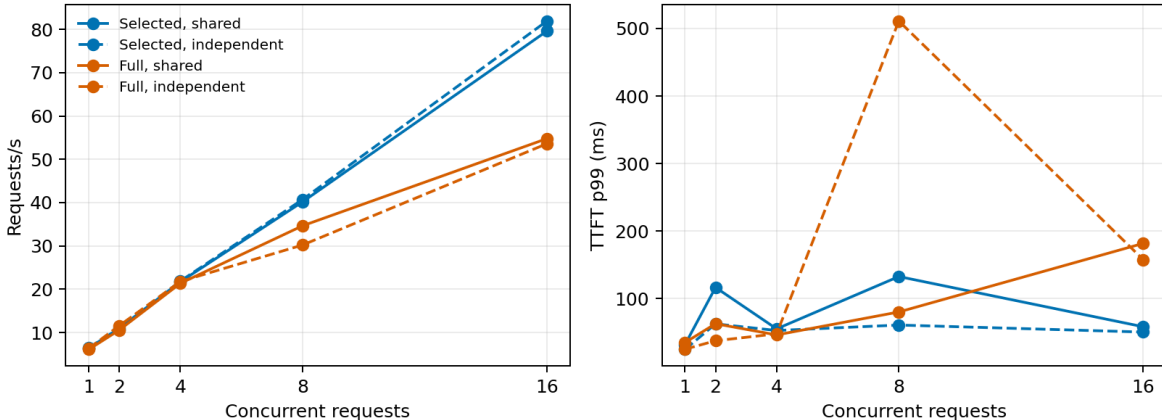


Figure 2: Expanded TensorRT-LLM E0 throughput and TTFT tails. Selected text retains its advantage through concurrency 16 for shared and independent resources.

The expanded sweep comprises 1,240 measured requests after explicit warmup. At concurrency 16, selected text reaches 79.6 requests/s with shared resources and 81.9 with independent resources. Full context reaches 54.8 and 53.6 requests/s, respectively. Selected TTFT p99 remains 50.5–58.4 ms, compared with 157.8–181.8 ms for full context. TensorRT-LLM reports 64–66 cached selected tokens and 1,039 cached full-context tokens. One independent full-context concurrency-eight group has a 511 ms TTFT p99 that does not persist at concurrency 16; we report it rather than smoothing a likely scheduler/cache disturbance out of the curve.

6.6 Context-pressure curve

We hold selected evidence at 67 prompt tokens and grow the full visible prompt from 648 to 1,040 and 1,276 tokens. Each point uses concurrency 16, 20 measured waves after explicit warmup, and identical answer-bearing evidence.

Selected throughput remains 78.0–81.9 requests/s and TTFT p99 remains 50.5–86.0 ms across all pressure points. Full-context throughput falls from 61.8–65.7 requests/s at 648 tokens to 52.6–56.9 at 1,276 tokens; TTFT p99 at the largest point is 149–173 ms. Unlike the vLLM independent-resource curve, TensorRT-LLM

Table 8: TensorRT-LLM context pressure at concurrency 16. All controlled outputs recover the target exactly.

Size	Workload	Context	Prompt tok.	Succ.	Req/s	Out tok/s	TTFT p99
Small	Shared	Selected	67	1.00	78.35	1567.0	66.7
Small	Independent	Selected	67	1.00	78.04	1560.8	86.0
Small	Shared	Full	648	1.00	61.84	1236.9	117.6
Small	Independent	Full	648	1.00	65.67	1313.5	111.8
Medium	Shared	Selected	67	1.00	79.61	1592.2	58.4
Medium	Independent	Selected	67	1.00	81.92	1638.5	50.5
Medium	Shared	Full	1040	1.00	54.77	1095.5	181.8
Medium	Independent	Full	1040	1.00	53.60	1072.0	157.8
Large	Shared	Selected	67	1.00	80.66	1613.2	57.7
Large	Independent	Selected	67	1.00	80.26	1605.2	58.0
Large	Shared	Full	1276	1.00	56.87	1137.3	148.5
Large	Independent	Full	1276	1.00	52.59	1051.8	172.6

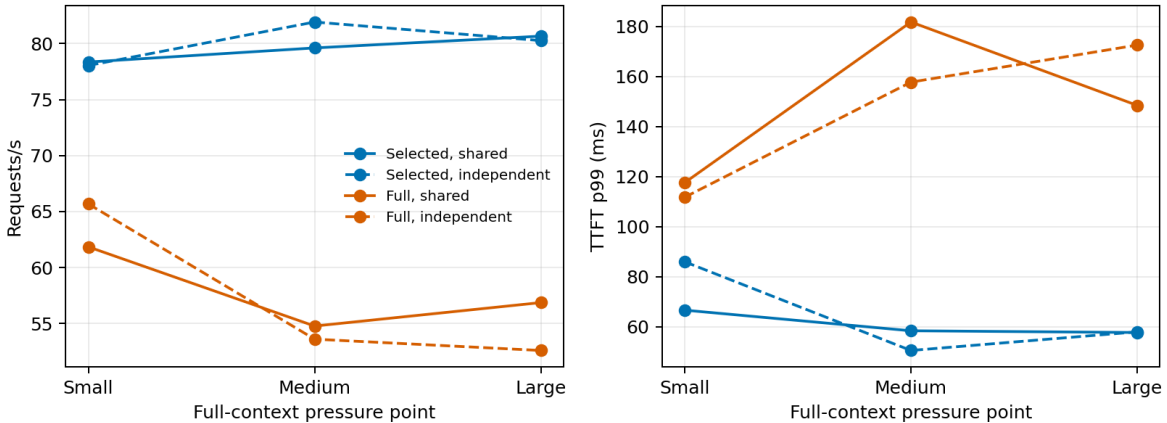


Figure 3: TensorRT-LLM peak-load throughput and TTFT tail as full-context pressure grows. Selection remains fixed at 67 prompt tokens.

does not collapse at the tested boundary. It instead shows a steadier quality-preserving cost increase. This remains selected-text E0 and does not establish native non-prefix K/V attachment or HBM savings.

On 2Wiki and HotpotQA, selected evidence improves token F1 over no context and answer containment reaches 0.25 and 0.20. Full context is strongest on Hotpot (F1 0.132), while selected is slightly stronger on 2Wiki (0.105 versus 0.095). QASPER remains difficult for this checkpoint: selected and full F1 are 0.069 and 0.087. Selected median TTFT is 23.5–28.7 ms, versus 37.2–63.3 ms for full context. These are natural input and answer metrics, but TinyLlama’s low absolute competence prevents a broad QA claim.

Table 9: Natural-QA E0 cohort: 20 examples per dataset and two repeats. Times are milliseconds; retrieval is frozen across execution conditions.

Dataset	Context	F1	Contain.	Prompt tok.	TTFT p50	TTFT p95
2Wiki	None	0.066	0.150	39	22.6	31.9
2Wiki	Selected	0.103	0.275	403	32.9	43.7
2Wiki	Full	0.094	0.300	940	45.6	106.3
hotpotqa	None	0.074	0.050	45	23.4	31.5
hotpotqa	Selected	0.095	0.200	438	37.7	46.2
hotpotqa	Full	0.123	0.350	1441	53.2	117.7
qasper	None	0.046	0.000	33	22.6	33.2
qasper	Selected	0.080	0.050	420	39.0	44.7
qasper	Full	0.067	0.100	1569	68.8	121.2

7 What Is Established

The experiment establishes that the shared PRA runtime can drive a TensorRT-LLM E0 endpoint without confusing semantic and prefix identity, and that smaller frozen selected-text inputs improve warm loaded serving on this hardware/model pair. It also establishes the exact point where a native integration must enter the engine. The connector is useful for storage and movement but is not itself the attention attachment mechanism.

The repeated-session cohort additionally establishes that session deduplication can preserve output behavior while reducing model-visible input. It does not establish task quality or a TTFT gain on this weak model and fully warm cache.

The experiment does not establish native K/V parity, native memory savings, connector hit economics, disaggregated transfer, quantized PRA K/V, TP=2, or native-E2 QA quality. Those claims require the missing non-prefix attention seam.

8 Required Upstream Native Contract

A supported implementation should add one request field containing authorized external block handles and source positions. During attention metadata construction, those blocks must remain outside prefix lookup, be pinned for the request lifetime, and participate with local K/V in one normalization. Crucially, the operator contract must preserve independent local write offsets and positional geometry; a Python-only block-table edit is not sufficient. The parity ladder should then compare: ordinary full prefix, geometry-matched cached source, full native source, sparse multi-resource source, and cached decode. Only after exact/logit parity is acceptable should connector transfer, eviction hints, disaggregation, and quantized K/V be benchmarked.

9 Negative Results and Limitations

The controlled code task is synthetic; the additional natural cohort is small and uses a weak QA checkpoint. The original tail estimates use five waves and the expanded endpoints use ten. Only one model, one GPU, one TensorRT-LLM version, and one topology are measured. The full context is bounded to the model's 2,048-token limit. Most importantly, all generation evidence is E0. The source audit is specific to package 1.2.1 and its recorded source hashes; later TensorRT-LLM releases may add a narrower supported contract and should be reaudited rather than assumed equivalent. The session-aware cohort is also limited to a synthetic two-turn task. Its 20/20 behavioral equivalence accompanies 0/20 exact task success, so it cannot stand in for a capable-model conversational evaluation.

10 Conclusion

TensorRT-LLM supplies the physical K/V machinery PRA should reuse. The current integration supplies semantic identity, safe lifecycle contracts, and a measured selected-text baseline. On the tested warm workload, smaller selected inputs improve throughput and TTFT under load. The installed public API does not yet expose arbitrary non-prefix K/V attachment, so native PRA remains a well-specified upstream integration task rather than a completed result. The installed-source audit further shows that implementing it correctly in 1.2.1 would require a coupled request, scheduler, metadata, lifecycle, and fused operator fork. That cost fails the paper's maintainability gate, so E0 selected text remains the recommended TensorRT-LLM path for this version.

Reproducibility

Raw JSON, generated tables and plots, environment audit, launch configuration, and experiment scripts are under `docs/papers/shared/results/paper6_4.tensorrt_llm` and `experiments/paper6_4.tensorrt_llm`. The repeated-session cohort uses `experiments/engine_serving/run_session_aware_selected_context.py`

against the live OpenAI-compatible endpoint with the five default seeds and four repetitions per seed. The native feasibility artifact is generated by `experiments/paper6_4_tensorrt_llm/run_native_seam_audit.py` against the live installed package; it records source hashes, line evidence, gate decisions, required patch surfaces, and rejected shortcuts.

References

- [1] J. Simao. *Productizing Progressive Retrieval Attention: A Runtime, Gateway, and Deployment Architecture*. 2026.
- [2] NVIDIA. *TensorRT-LLM Documentation*.
<https://nvidia.github.io/TensorRT-LLM/>
- [3] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023.